

forge Framework

Informe de Análisis Técnico v2.0

Análisis Adversarial Comparativo v1 → v2

Análisis Adversarial Independiente

3 de mayo de 2026

Índice

1. Resumen ejecutivo	3
2. Introducción: contexto y metodología	3
2.1. El framework	3
2.2. Metodología	3
3. Problemas del v1 resueltos en v2	4
3.1. Bug de <code>install_agent</code> — resuelto	4
3.2. Flag <code>--only</code> inexistente — resuelto	4
3.3. Adapters de OpenCode y Kiro vacíos — parcialmente resuelto	4
3.4. Ausencia de teardown — resuelto	4
3.5. Ecosistema de profiles insuficiente — resuelto	5
3.6. Otras mejoras verificadas	5
4. Problemas del v1 que persisten en v2	5
4.1. Fix messages del audit no ejecutables	5
4.2. Mutación silenciosa en el hook pre-commit	5
4.3. Similitud de texto como métrica de calidad	5
4.4. Mecanismo de actualización destructivo	6
5. Problemas nuevos detectados en v2	6
5.1. <code>agent-standard.md</code> desincronizado	6
5.2. <code>fullstack-engineer</code> sin descripción en <code>forge-init.py</code>	6
5.3. <code>-tool opencode</code> no invoca el adapter de OpenCode	6
5.4. Brechas críticas en el suite de tests	6
6. Nuevas fortalezas presentes solo en v2	7
6.1. Suite de tests con 112 casos	7
6.2. Adapters con lógica condicional correcta	7
6.3. Test paramétrico de profiles como garantía de estándar	7
6.4. <code>forge-teardown.py</code> con lógica deliberada	7
7. Análisis comparativo v1 vs. v2	7

8. Conclusiones y recomendaciones	7
8.1. Recomendaciones por perfil de equipo	8
8.2. Hoja de ruta recomendada para v3	8
Anexo A: Bugs nuevos encontrados en v2	8

1. Resumen ejecutivo

forge es un framework de agentic development que centraliza la configuración de agentes de IA en un único archivo `project.yaml` y genera, mediante scripts determinísticos, los artefactos de configuración para múltiples runtimes (Claude Code, OpenCode, Kiro). Su propuesta arquitectónica —taxonomía de tres tiers, compliance por diseño, Spec-Driven Development como workflow obligatorio— se mantiene intacta desde v1 y está respaldada por evidencia de código.

La segunda ronda de análisis (v2) evalúa el commit `d828157`, que incorpora mejoras directamente motivadas por el análisis v1. Las conclusiones son mixtas. En las dimensiones que el análisis v1 identificó como críticas, la mejora es real y verificable: el bug de `install_agent` está corregido, los adapters de OpenCode y Kiro tienen implementaciones funcionales, el framework tiene teardown, el ecosistema de profiles pasó de cuatro a ocho, y existe una suite de 112 tests. En las dimensiones donde los problemas son estructurales —coherencia entre documentación y código, integración real del flujo multi-runtime, ejecutabilidad de los fix messages del audit— las mejoras son parciales o nulas.

La recomendación consolidada es: forge v2 es adoptable para equipos de 3-8 personas que usen Claude Code como runtime principal, que entiendan que la documentación central `agent-standard.md` no refleja el estado del código, y que no dependan de los fix messages del audit como comandos ejecutables.

2. Introducción: contexto y metodología

2.1. El framework

forge se instala como git submodule en proyectos de desarrollo con agentes de IA. Su función es proporcionar una base de agentes especializados, skills componibles, y herramientas de auditoría que hagan predecible y auditable el comportamiento de los agentes en producción.

La arquitectura central se basa en tres tiers:

- **Tier 1 (Universal):** agentes que aplican a cualquier proyecto independientemente del stack.
- **Tier 2 (Profile):** agentes especializados en un stack técnico específico (FastAPI, Rails, NestJS, etc.).
- **Tier 3 (Dominio):** agentes creados por el equipo para necesidades específicas del proyecto.

El `project.yaml` actúa como fuente de verdad: stack, roster de agentes, frameworks de compliance, fases del sprint. Scripts determinísticos generan los artefactos de configuración para cada runtime desde esa única fuente.

2.2. Metodología

Este informe consolida dos análisis adversariales independientes realizados sobre el mismo commit:

- **Análisis favorable** (`advocate-report.md v2`): evalúa el diseño, las fortalezas, y las mejoras concretas incorporadas desde v1.
- **Análisis crítico** (`critic-report.md v2`): examina la implementación en busca de problemas que afecten la adopción en producción.

Ambos análisis se basan en lectura directa del código fuente: scripts Python, agentes Markdown, tests, hooks, adapters, y documentación. No se ejecutó el framework en un proyecto real.

3. Problemas del v1 resueltos en v2

3.1. Bug de `install_agent` — resuelto

El análisis v1 identificó que la función `install_agent()` en `forge-init.py` retornaba siempre `UPDATE` para instalaciones nuevas, porque evaluaba `dst.exists()` después de copiar el archivo. La corrección consiste en capturar el estado del archivo antes de la operación de copia:

```
1 # evaluar ANTES de copiar
2 already_existed = dst.exists()
3 shutil.copy2(src, dst)
4 return "UPDATE" if already_existed else "OK"
```

Listing 1: Corrección en `forge-init.py` (líneas 95–105)

El test `test_ok_no_es_update_en_instalacion_nueva` documenta explícitamente el bug original en su docstring. Corrección completa.

3.2. Flag `--only` inexistente — resuelto

El audit generaba fix messages con `--force --only=<agente>`, pero el flag `--only` no existía. Ahora está implementado en ambas sintaxis (`--only=nombre` y `--only nombre`) con cobertura de tests en dos archivos distintos.

3.3. Adapters de OpenCode y Kiro vacíos — parcialmente resuelto

Los directorios `adapters/opencode/` y `adapters/kiro/` estaban vacíos en v1. Ahora contienen implementaciones funcionales con 24 tests de integración. La calificación es parcialmente resuelto porque el flag `--tool opencode` de `forge-init.py` no invoca el adapter (ver sección 5.3).

3.4. Ausencia de `teardown` — resuelto

`forge-teardown.py` no existía en v1. Ahora existe con dry-run por defecto, eliminación selectiva de artefactos de forge (no Tier 3 del proyecto), y 8 tests de cobertura.

3.5. Ecosistema de perfiles insuficiente — resuelto

En v1 había cuatro perfiles. En v2 hay ocho: se agregaron `fastapi`, `express`, `rails`, `nestjs`. El test paramétrico en `test_profiles.py` verifica que cada agente de cada perfil cumpla el estándar de frontmatter, modelo, secciones y longitud mínima.

3.6. Otras mejoras verificadas

- **Fases de sprint en CLAUDE.md:** `_render_phases()` lee `sprint.phases` desde `project.yaml` y genera el listado real.
- **Disclaimer en compliance:** la sección “Limitaciones” advierte que el agente opera sobre conocimiento de entrenamiento, no texto legal oficial.
- **Compatibilidad Python 3.9:** se usa `Optional[str]` en lugar de `str | None`.

4. Problemas del v1 que persisten en v2

4.1. Fix messages del audit no ejecutables

El audit genera la siguiente cadena como acción correctiva:

```
1 "fix": f"forge-init.py --tool claude-code --force --only={agent['name']}"
```

Listing 2: `forge-audit.py` líneas 223, 242

`forge-init.py` no está en el PATH del sistema. El README muestra el comando correcto: `python3 .agentic/scripts/forge-init.py --tool claude-code`. Un usuario que copie y ejecute la acción correctiva del audit recibirá `command not found`. Ningún test verifica el formato de los fix messages.

4.2. Mutación silenciosa en el hook pre-commit

El hook sigue haciendo `git add` sobre `docs/progress.html` dentro del proceso de commit, sin que el desarrollador haya inspeccionado ese archivo:

```
1 if ! git -C "$ROOT" diff --quiet "$HTML"; then
2   echo "[forge pre-commit] Actualizando docs/progress.html..."
3   git -C "$ROOT" add "$HTML"
4 fi
```

Listing 3: `hooks/pre-commit`: mutación activa

El mensaje informativo agregado en v2 notifica la operación pero no cambia el comportamiento. No existe ningún test que cubra el hook ni `token-stats.py`.

4.3. Similitud de texto como métrica de calidad

El audit usa `SequenceMatcher.ratio()` con umbrales fijos (`SIMILARITY_WARN` = 0.80, `SIMILARITY_OUTDATED` = 0.50). Un agente correctamente especializado para Rails puede tener baja similitud con el core genérico y aparecer como posiblemente desactualizado.

4.4. Mecanismo de actualización destructivo

`--force` sobrescribe el agente de destino completamente. No existe merge, diff interactivo, ni versionado de personalizaciones locales.

5. Problemas nuevos detectados en v2

5.1. agent-standard.md desincronizado

La documentación de referencia de profiles contiene:

```
1 | 'rails' | 'backend-engineer' *(pendiente)* |
2 | 'fastapi' | 'backend-engineer' *(pendiente)* |
```

Listing 4: agent-standard.md: datos incorrectos

Los agentes reales son `fullstack-engineer` (rails) y `api-engineer` (fastapi). Los profiles `express` y `nestjs` no aparecen en la tabla en absoluto.

5.2. fullstack-engineer sin descripción en forge-init.py

El profile rails provee `fullstack-engineer`, pero el diccionario `role_descriptions` en `forge-init.py` no tiene esa entrada. Un proyecto que use el profile rails verá “Agente de implementación” (descripción genérica) en su `AGENTS.md` generado. El adapter de Kiro sí tiene la entrada correcta, creando una inconsistencia entre adapters para el mismo agente.

5.3. --tool opencode no invoca el adapter de OpenCode

`forge-init.py` agrupa `claude-code` y `.opencode` bajo el mismo bloque de código:

```
1 if tool in ("claude-code", "opencode", "all"):
2     label = "Claude Code / OpenCode" if tool == "all" else tool.
      title()
3     init_claude_code(root, forge, config)    # mismo codigo para
      ambos
4     install_claude_commands(root, forge, config)
5     init_wiki(root, forge, config)
```

Listing 5: forge-init.py líneas 382–389: mismo código para ambos tools

Un usuario de OpenCode que ejecute `forge-init.py --tool opencode` obtendrá una instalación de Claude Code (`.claude/agents/`, slash commands) con exit code 0 y sin mensajes de error. El adapter `adapters/opencode/generate-agents-md.py` no se invoca desde `forge-init.py` en ningún caso.

5.4. Brechas críticas en el suite de tests

El suite no incluye:

- Tests para `token-stats.py` (único script sin cobertura).
- Tests para el hook pre-commit.

- Tests que validen el formato de los fix messages del audit.
- Tests que detecten inconsistencias entre `agent-standard.md` y el código.
- Tests que ejecuten `--tool opencode` y verifiquen output específico de OpenCode.

6. Nuevas fortalezas presentes solo en v2

6.1. Suite de tests con 112 casos

La adición más significativa para la madurez del proyecto. Cuatro categorías complementarias: tests de integración que ejecutan scripts reales sobre directorios temporales, tests unitarios con `importlib` controlando `sys.argv`, tests estructurales paramétricos que recorren todos los profiles, y tests de regresión con docstrings que documentan los bugs originales. El `confest.py` tiene fixtures con deep merge que permiten configurar exactamente lo necesario sin boilerplate.

6.2. Adapters con lógica condicional correcta

El adapter de Kiro genera `compliance.md` solo si hay frameworks configurados en `project.yaml`. Los tres adapters implementan la misma lógica de compliance automático. El adapter de OpenCode lee descriptions desde el frontmatter de los agentes, respetando la prioridad `profiles > core`.

6.3. Test paramétrico de profiles como garantía de estándar

`test_profiles.py` recorre dinámicamente todos los profiles y verifica frontmatter completo, modelo correcto, secciones requeridas y longitud mínima. Agregar un profile sin que pase estos tests es imposible sin romper el suite.

6.4. `forge-teardown.py` con lógica deliberada

La distinción entre lo que le pertenece al framework y lo que le pertenece al proyecto está codificada en la lógica del script. Los agentes Tier 3 sobreviven el teardown.

7. Análisis comparativo v1 vs. v2

8. Conclusiones y recomendaciones

`forge v2` resolvió los problemas más visibles identificados en v1. Este es un mérito real que el análisis debe reconocer. Sin embargo, el patrón que emerge al examinar los problemas nuevos es preocupante: al agregar funcionalidad (profiles, adapters, scripts), la coherencia interna del framework se degradó. La documentación de referencia ahora describe un ecosistema más desactualizado que en v1. El adapter de OpenCode existe pero está desconectado del flujo principal. Un agente nuevo (`fullstack-engineer`) no está registrado en el script principal.

Cuadro 1: Comparativa de dimensiones clave entre v1 y v2

Dimensión	v1	v2	Tendencia
Profiles implementados	4	8	↑
Tests	0	112	↑
Bug <code>install_agent</code>	Presente	Resuelto	
Flag <code>--only</code>	Inexistente	Implementado	
Adapter OpenCode	Vacío	Funcional (no integrado)	~
Adapter Kiro	Vacío	Funcional (integrado)	
Teardown	Ausente	Presente (parcial)	↑
Fix messages ejecutables	No	No	×
Hook pre-commit (mutación)	Silenciosa	Con notificación	~
Disclaimer en compliance	Ausente	Presente	
<code>agent-standard.md</code> sincronizado	Parcialmente	Menos que v1	↓
<code>--tool opencode</code> invoca adapter	N/A	No	×
<code>fullstack-engineer</code> en <code>role_descriptions</code>	N/A	Ausente	×
Python 3.9 compatible	No	Sí	

8.1. Recomendaciones por perfil de equipo

Adopción inmediata Equipos de 3–8 personas con Claude Code como runtime principal, stack cubierto por los ocho profiles, y disposición a complementar la documentación con lectura del código fuente.

Esperar Equipos que dependan de la documentación oficial, usen OpenCode como runtime principal, o necesiten que los fix messages del audit sean ejecutables sin modificación.

Contribuir Equipos cuyo stack no está en los 8 profiles. El test paramétrico facilita agregar profiles sin overhead adicional.

8.2. Hoja de ruta recomendada para v3

1. Sincronizar `agent-standard.md` con el ecosistema real de profiles.
2. Hacer que `--tool opencode` invoque el adapter correcto.
3. Corregir el formato de los fix messages del audit para que sean ejecutables.
4. Agregar tests que detecten inconsistencias entre capas (documentación, scripts, adapters).

Anexo A: Bugs nuevos encontrados en v2

A.1 Fix messages no ejecutables en el audit

Archivo: `scripts/forge-audit.py`, líneas 223 y 242

```
1 "fix": f"forge-init.py --tool claude-code --force --only={agent['name']}"
```

Listing 6: Fix message actual — no ejecutable

`forge-init.py` no está en el PATH del sistema. El comando correcto es `python3 .agentic/scripts/forge-init.py --tool claude-code`. Un usuario que copie y ejecute el fix sugerido recibirá `command not found`.

Fix sugerido:

```
1 "fix": f"python3 .agentic/scripts/forge-init.py --tool claude-code --force --only={agent['name']}"
```

Listing 7: Fix message corregido

A.2 fullstack-engineer sin entrada en role_descriptions

Archivo: `scripts/forge-init.py`, diccionario `role_descriptions`

El profile rails provee `fullstack-engineer`, pero ese nombre no tiene entrada en `role_descriptions`. El `AGENTS.md` generado mostrará `‘gente de implementación’` para ese agente.

Fix sugerido:

```
1 "fullstack-engineer": "Full-stack -- frontend, backend y base de datos en proyectos Rails"
```

Listing 8: Entrada faltante en role_descriptions

A.3 --tool opencode no invoca el adapter de OpenCode

Archivo: `scripts/forge-init.py`, bloque de selección de tool (líneas 382–389)

Ejecutar `forge-init.py --tool opencode` produce exactamente el mismo resultado que `--tool claude-code`. El adapter `adapters/opencode/generate-agents-md.py` no se invoca desde `forge-init.py` en ningún caso. El script termina con exit code 0.

Fix sugerido: agregar un branch específico para `tool == .opencode` que invoque el adapter vía subprocess, siguiendo el mismo patrón que `--tool kiro` usa para invocar `adapters/kiro/generate-steering.py`.